Esercizio 3 (11 punti)

Si consideri l'architettura RISC-V con pipeline. Il programma qui di seguito effettua la somma dei soli valori pari (dopo averli divisi per due) e degli elementi dispari presenti nell'array vettore di lunghezza 10. Infine, viene stampato il valore di tale somma.

```
1  .data
2  vettore: .word 24, 1, 46, 54, 50, 12, 2, 11, 39, 4
3   # 7 pari, 3 dispari
4
5  .text                        # Sommo solo i dispari moltipl. per 2
6  main:   addi t0, zero, 0     # somma parziale pari
7          auipc s0, 0          # operazione di tipo lui che fa s0 = PC + 0
8          addi s0, s0, 1020    # s0 ora contiene l'indirizzo base del vettore
9          addi t1, zero, 0     # somma parziale dispari
10         addi t4, s0, 40      # primo indirizzo fuori dall'array
11 ciclo:  lw   t3, 0(s0)
12         andi t2, t3, 1       # è dispari?
13         bne  t2, zero, salta # se lo è, salto
14         srli t3, t3, 1       # altrimenti divido per 2 il numero
15         add  t0, t0, t3      # e lo accumulo
16         addi t3, zero, 0   # azzera il registro per non sommarlo con i dispari
17 salta:  add t1, t1, t3       # accumula dispari
18         addi s0, s0, 4       # prossimo elemento
19         blt  s0, t4, ciclo   # fine del ciclo?
20
21         add t0, t0, t1
22         addi a7,x0, 1        # stampa…
23         mv a0, t0            # … la somma
24         ecall
```

Si supponga che non si faccia uso di alcuna pseudoistruzione, e che le ecall non richiedano alcuno stallo. Si consideri che la decisione di branch venga presa in fase di ID, e che le unità di forwarding (quando usate) includano il forwarding da EXE a EXE, da MEM a EXE, e (per il branch) da EXE a ID.

Si indichino (ignorando hazard che possano concernere la ecall):

1) Per i primi 16 cicli di clock (**senza unità di forwarding**), le istruzioni tra le quali sono presenti **data hazard** – indicare i numeri di linea N ed M (visibili alla sinistra delle linee di codice in figura) ed il registro coinvolto xY (nel formato N / M / xY, ad esempio 17 / 18 / t0 se c'è un data hazard causato dal registro t0 tra l'istruzione alla linea 17 e quella alla linea 18);

7/8/s0
8/10/s0
11/12/t3
12/13/t2

2) Per i primi 22 cicli di clock (**con unità di forwarding**) le istruzioni tra le quali sono presenti **control hazard** – indicare i numeri di linea N ed M (nel formato N / M, ad esempio se l'istruzione alla linea 17 causa un control hazard con l'istruzione alla linea 23 scrivere: 17 / 23);
19/11

3) quanti **cicli di clock** sono necessari ad eseguire **con unità di forwarding**:
   a) Riempire la tabella con i dati richiesti:

sezione	numero di colpi di clock richiesti
pre-ciclo	5
1° Ciclo	13 (composto da 9 istruzioni, 3 data hazard e un control hazard)
2° Ciclo	11 (composto da 7 istruzioni, 2 data hazard e 2 control hazard)
uscita ciclo e post ciclo	7 (contando il riempimento della pipeline e un control hazard in meno dell'ultimo ciclo)
Totale	136

numero di colpi di clock del ciclo di programma si intende il numero di colpi di clock che trascorrono dal fetch della prima istruzione del ciclo al fetch della stessa istruzione esclusa

b) il **primo** ciclo spiegando il calcolo effettuato
13
c) b) **tutto** il programma spiegando il calcolo effettuato;
136

5+13*7+11*3+7=136 (l'ultimo ciclo ha un control hazard in meno)

Un altro modo di contarlo sarebbe
4 caricamento pipeline
5 istruzioni pre-ciclo
13 istruzioni ciclo pari tranne l'ultimo (composto da 9 istruzioni, 3 data hazard e un control hazard)
11 istruzioni cicli dispari (composto da 7 istruzioni, 2 data hazard e 2 control hazard)
12 istruzioni ultimo ciclo (composto da 9 istruzioni, 3 data hazard e no control hazard)
4 istruzioni uscita

4) quanti **cicli di clock** sarebbero necessari ad eseguire **senza forwarding**:
a) Riempire la tabella con i dati richiesti:

sezione	cicli di clock richiesti
pre-ciclo	8
1° Ciclo	20
2° Ciclo	14
uscita ciclo e post ciclo e riempimento pipeline	8
Totale	

numero di colpi di clock del ciclo di programma si intende il numero di colpi di clock che trascorrono dal fetch della prima istruzione del ciclo al fetch della stessa istruzione esclusa

b) il **secondo** ciclo spiegando il calcolo effettuato

20

c) **tutto** il programma spiegando il calcolo effettuato;

198

8+20*7+14*3+8=198

5) quali sono, per ognuna delle cinque fasi, le **istruzioni** (o le bolle) in pipeline durante il **15° ciclo di clock** (**con forwarding**);

IF: 18. addi s0, s0, 4

ID: 17. add t1, t1, t3

EX: 16. addi t3, x0, 0

MEM: 15. add t0, t0, t3

WB: 14. srli t3, t3, 1

6) Domanda teorica